

PRACTICAL PROJECT REPORT

Smart Campus Help Desk

Implementing Continuous Delivery and a Blue-Green Deployment Strategy

Module	AZ-400 Designing and Implementing Microsoft DevOps Solutions
Focus	Module 5 — Continuous Delivery and Deployment Strategies
Student	Gregory Candiotes Student no. 20250656
Institution	CTU Training Solutions
Project	SmartCampus-DevOps
Application	SmartCampusHelpDesk (ASP.NET Core 8)
Repo reference	[PASTE Azure Repos URL HERE]
Cost constraint	Azure DevOps free organisation + local/Docker Blue-Green environments. No paid Azure App Service used.

1. Introduction

The institution currently releases Help Desk changes straight into production. That practice exposes students to downtime when a release fails. This practical implements a controlled DevOps workflow: plan on Azure Boards, develop in Git feature branches, review through pull requests, integrate with an Azure Pipelines CI pipeline (Restore → Build → Test → Publish), and release with a Blue-Green strategy so production traffic moves only after the candidate environment has been validated. The previous environment is retained for rollback.

Because the brief allows free-tier and student resources, Azure DevOps Services is used for Boards, Repos and Pipelines (free for a private project). The two runtime environments are provided locally with Docker: Blue on port 8081 (Version 1.0) and Green on port 8082 (Version 2.0). An nginx proxy on port 8080 represents production traffic. Switching the proxy is the Blue-Green cutover; restoring the Blue upstream is rollback. This preserves the required workflow without paid Azure compute.

2. Implemented workflow

Plan → Develop → Commit → Review → Build → Test → Deploy → Validate → Switch → Rollback

Azure Boards feeds Azure Repos. Feature work is isolated on a feature branch, reviewed by pull request into develop, then integrated to main. A commit on main triggers Continuous Integration. The published artifact is deployed to the inactive colour. After validation, production traffic is switched. The idle colour remains available.

3. Task 1 — Azure Boards planning

Project name: **SmartCampus-DevOps**. Work items were structured so the business need (a reliable campus Help Desk) parents the technical work.

Type	Title	Relationship
Epic	Smart Campus Help Desk	Overall project
Feature	Controlled development and Blue-Green deployment	Child of Epic
User Story US-01	Capture a campus support request	Child of Feature
User Story US-02	View submitted support requests	Child of Feature
User Story US-03	Source control and branching	Child of Feature
User Story US-04	Pull request review workflow	Child of Feature
User Story US-05	Continuous Integration pipeline	Child of Feature
User Story US-06	Blue-Green release and rollback	Child of Feature

Figure 1 placeholder: paste Azure Boards screenshot showing Epic → Feature → User Stories → Tasks.

Tasks under the stories include: model and form, issues table, Git remotes and branches, PR review, pipeline YAML, failed-build investigation, Blue deploy, Green deploy, validation, switch and rollback.

4. Tasks 2–3 — Application Version 1.0 (BLUE)

SmartCampusHelpDesk is an ASP.NET Core 8 Razor Pages application. Students capture Student Name, Issue Category (Network, Computer, Software, Account, Other), Description, Status and Date. Submitted requests are listed on the same page. State is held in an in-memory store, which is sufficient for the DevOps focus of the practical.

Version and colour are not hard-coded in every view. They come from configuration so the same build can run as Blue or Green:

AppInfo: Version 1.0, EnvironmentName BLUE, EnhancedUi false. The layout banner renders this as **Application Version: 1.0 | Environment: BLUE.**

Figure 2 placeholder: local run of Version 1.0 showing the BLUE banner and a submitted issue.

5. Tasks 4–5 — Source control and pull requests

Git is initialised in the project root. The remote is Azure Repos inside SmartCampus-DevOps. Branching strategy:

- **main** — releasable code. CI trigger.
- **develop** — integration branch.
- **feature/helpdesk-v1** and later **feature/helpdesk-v2** — all application changes.

No work is committed directly to main. A change on the feature branch is pushed, a pull request targets develop, the diff is reviewed, and the PR is completed only after review. A second PR promotes develop to main to trigger CI.

Figure 3 placeholder: Azure Repos branches. Figure 4 placeholder: completed PR with source and target branches.

6. Tasks 6–7 — Continuous Integration and failure investigation

azure-pipelines.yml runs on the Microsoft-hosted ubuntu-latest agent (included in the Azure DevOps free grant). Stages:

- **Restore** — NuGet restore for the web and test projects.
- **Build** — compile Release.
- **Test** — xUnit tests (store IDs, sort order, model validation).
- **Publish** — zip the web app and publish the drop artifact.

The pipeline is triggered from main. For Task 7 a deliberate compile error is introduced on a feature branch (invalid assignment in Program.cs), pushed, and the failed logs are captured. The error is then removed, the correction is committed, and the pipeline is confirmed green. The fault is never introduced on a live production process outside this practical.

Figure 5 placeholder: failed CI run and log line. Figure 6 placeholder: successful CI run producing the drop artifact.

7. Task 8 — Deploy Version 1.0 to Blue

Blue is the standing production colour for Version 1.0. Using Docker Compose, the blue service listens on port 8081 with AppInfo Version=1.0 and EnvironmentName=BLUE. The production proxy on port 8080 initially forwards to Blue. Users therefore see Version 1.0 / BLUE. Architecture at this point: Users → Production proxy → Blue → Version 1.0.

Figure 7 placeholder: Version 1.0 reachable in the Blue environment.

8. Tasks 9–11 — Version 2.0 and the Green environment

Version 2.0 is developed on feature/helpdesk-v2. The identifiable improvement is an enhanced homepage: campus coverage and support-hour cards, plus search across submitted issues. Configuration becomes Version 2.0, EnvironmentName GREEN, EnhancedUi true. The banner is green and reads **Application Version: 2.0 | Environment: GREEN**.

After PR review and a successful CI run against the Version 2 source, Green is started on port 8082. Blue is not removed. Architecture before cutover: BLUE — Version 1.0 — current production. GREEN — Version 2.0 — candidate release. Green is independently reachable for testing.

Figure 8 placeholder: both environments available — Blue 1.0 and Green 2.0.

9. Task 12 — Validate Green

Validation requirement	Expected	Result
Application starts successfully	Pass	Pass
Homepage loads correctly	Pass	Pass
Report Issue functionality operates	Pass	Pass
Submitted issues can be viewed	Pass	Pass
Application version	2.0	2.0
Environment	GREEN	GREEN
v2 UI improvement visible	Cards + search	Pass

Table 1. Green environment validation. No critical test failed, so promotion was approved.

10. Task 13 — Blue-Green production switch

Before the switch, production traffic on port 8080 targets Blue (v1.0). After validation, `deploy/switch-to-green.sh` replaces the nginx upstream with Green and reloads the proxy. Users now receive Version 2.0 / GREEN. Blue is left running as standby. After: Users → GREEN → Version 2.0. BLUE → Version 1.0 — standby.

Figure 9 placeholder: production URL after switch showing Version 2.0 GREEN.

11. Task 14 — Rollback

A critical defect is assumed in Version 2.0 after promotion. Rollback is `deploy/rollback-to-blue.sh`: the proxy upstream is restored to Blue and nginx is reloaded. Green stays available for investigation but is no longer production. Users again see Version 1.0 / BLUE. Expected path: Green 2.0 → problem identified → rollback → Blue 1.0.

Figure 10 placeholder: production URL after rollback showing Version 1.0 BLUE.

12. Practical reflection

12.1 Purpose of Continuous Integration

Continuous Integration compiles every accepted change on main through the same Restore → Build → Test → Publish path. Broken code is stopped before it becomes a deployment artifact. In this solution the pipeline is the quality gate between a completed pull request and any Blue or Green release. Version 2.0 is not deployed if CI fails, which is the control the institution asked for.

12.2 Why pull requests matter with multiple developers

A pull request makes the proposed diff visible before it lands on a shared branch. Reviewers can reject incomplete work, conflicting changes, or secrets committed by mistake. The source and target branches are recorded, which gives an audit trail. Without PRs, several developers pushing to the same branch overwrite or break each other and there is no mandatory review point.

12.3 Difference between Blue and Green

Blue and Green are two complete, independently running environments of the same application. At any moment only one colour receives production traffic. In this practical Blue hosts the known-good Version 1.0 and Green hosts the candidate Version 2.0. They do not overwrite each other. Promotion is a traffic switch, not an in-place overwrite of production files.

12.4 Why Version 2.0 was tested on Green first

Green is isolated from live users. Starting the app, loading the homepage, submitting an issue, viewing the list, and confirming version 2.0 / GREEN can all fail without taking the Help Desk away from students. Only after every required check passed was the release approved. Testing in production would have re-introduced the original operational risk.

12.5 How Blue-Green reduces deployment risk and downtime

The new build is brought up beside the live build. Cutover is a proxy (or load-balancer) change, which is fast compared with stopping the only running instance and copying new files over it. If the candidate misbehaves, traffic is pointed back at the untouched colour. Students keep a working Help Desk instead of sitting through a failed in-place deploy.

12.6 Why the previous Blue environment is kept

Rollback only works if the last known-good environment still exists. Keeping Blue running after the switch means Version 1.0 can be restored in seconds without rebuilding or redeploying. If Blue had been deleted at

cutover, a defect in 2.0 would have forced an emergency rebuild and a longer outage.

13. Evidence checklist

Required evidence	Where captured
Azure Boards Epic/Feature/Stories/Tasks	Screenshot — insert after section 3
Azure Repos + branches main/develop/feature	Screenshot — insert after section 5
Completed pull request	Screenshot — insert after section 5
Failed CI pipeline + log cause	Screenshot — insert after section 6
Successful CI pipeline + artifact	Screenshot — insert after section 6
Version 1.0 in Blue	http://localhost:8081 screenshot
Version 2.0 in Green	http://localhost:8082 screenshot
Validation results	Table 1 in this report
Production switch to Green 2.0	http://localhost:8080 after switch
Rollback to Blue 1.0	http://localhost:8080 after rollback
Azure Repos reference	Cover page — paste URL

14. Conclusion

The practical demonstrates a full controlled-delivery path for the Smart Campus Help Desk. Planning, source control, review, CI and Blue-Green release replace direct-to-production deploys. Version 2.0 only becomes production after a green pipeline and a passing validation table. Version 1.0 remains on Blue so a defect can be reversed without rebuilding. The same workflow can later be pointed at two free Azure Web Apps if an Azure student subscription is available; the pipeline and branching model do not change.